Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты группы 25ВВВ2
Ионова А.Н.
Прокофьева А.А.

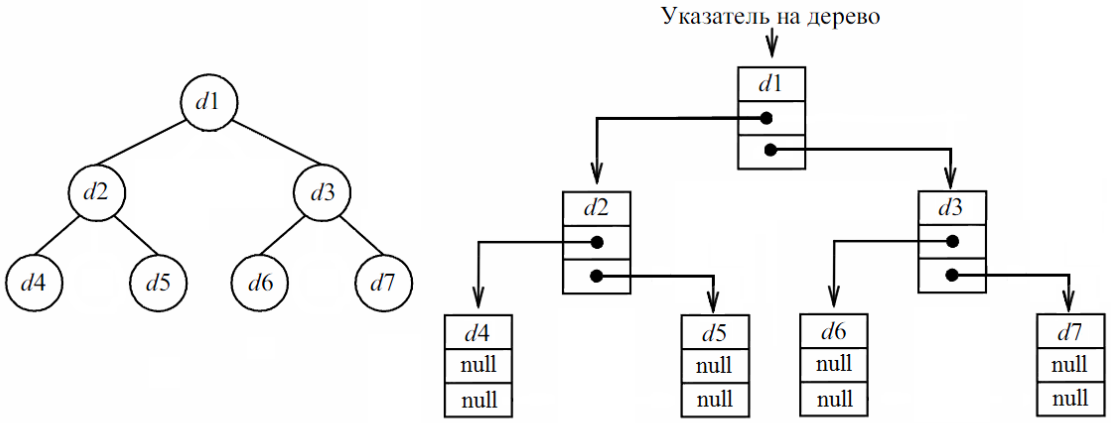
Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Бинарное дерево поиска.

Бинарные деревья – это деревья, у каждого узла которого возможно наличие только двух сыновей. Двоичные деревья являются упорядоченными.



Двоичное дерево можно представить в виде нелинейного связанного списка.

Бинарное дерево поиска — это бинарное дерево, обладающее дополнительными свойствами:

– значение левого потомка меньше значения родителя;

– значение правого потомка больше значения родителя.

Такие структуры используются для сохранения данных в отсортированном виде.

В простейшем случае для создания элемента списка используется структура, в которой объединяются полезная информация и ссылка на следующий элемент списка:

Задание 1.

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Функция поиска (FindNode)

```
ФУНКЦИЯ FindNode(r, data):
    ЕСЛИ r = NULL ТО
        ВЕРНУТЬ NULL
    КОНЕЦ ЕСЛИ

    ЕСЛИ r.data = data ТО
        ВЕРНУТЬ r
    КОНЕЦ ЕСЛИ

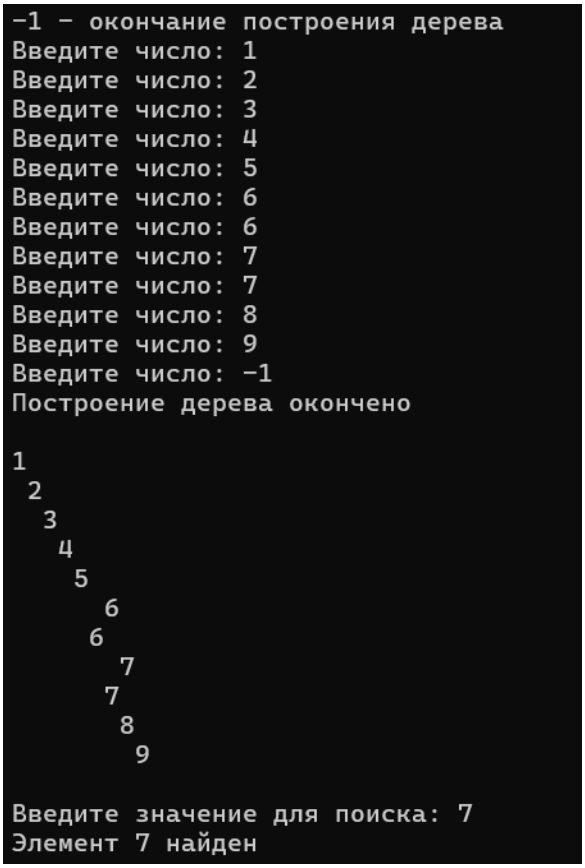
    ЕСЛИ data > r.data ТО
        ВЕРНУТЬ FindNode(r.left, data)
    ИНАЧЕ
        ВЕРНУТЬ FindNode(r.right, data)
    КОНЕЦ ЕСЛИ
КОНЕЦ ФУНКЦИИ
```

Фрагмент главной программы (поиск)

```
ВЫВЕСТИ "Введите значение для поиска: "
СЧИТАТЬ число D

found = FindNode(root, D)
ЕСЛИ found ≠ NULL ТО
    ВЫВЕСТИ "Элемент ", found.data, " найден"
ИНАЧЕ
    ВЫВЕСТИ "Элемент ", D, " в дереве не найден"
КОНЕЦ ЕСЛИ
```

Результат выполнения программы



Задание 2

Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Функция подсчёта (CountOccurrences)

```
ФУНКЦИЯ CountOccurrences(r, data):
    ЕСЛИ r = NULL ТО
        ВЕРНУТЬ 0
    КОНЕЦ ЕСЛИ

    count = 0
    ЕСЛИ r.data = data ТО
        count = 1
    КОНЕЦ ЕСЛИ

    count = count + CountOccurrences(r.left, data)
    count = count + CountOccurrences(r.right, data)

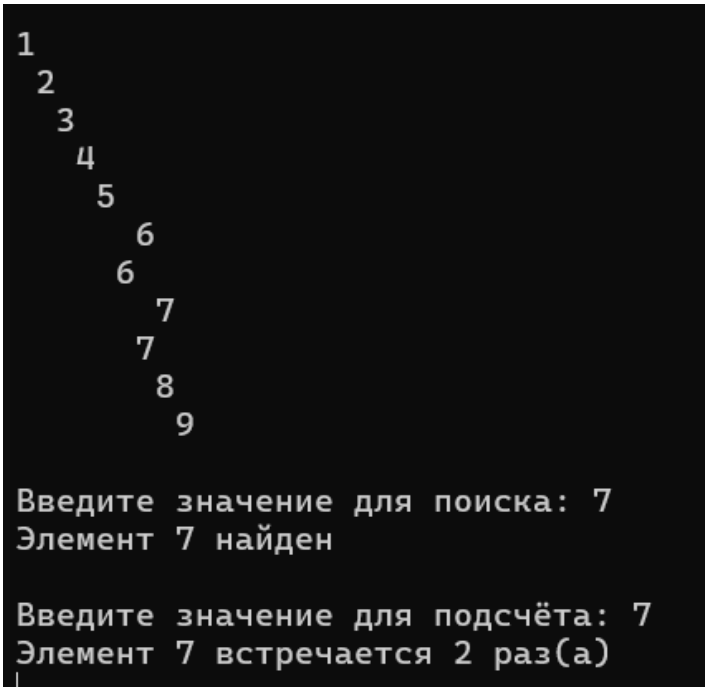
    ВЕРНУТЬ count
КОНЕЦ ФУНКЦИИ
```

Фрагмент главной программы (подсчёт)

```
ВЫВЕСТИ "Введите значение для подсчёта: "
СЧИТАТЬ число D

cnt = CountOccurrences(root, D)
ЕСЛИ cnt = 0 ТО
    ВЫВЕСТИ "Элемент ", D, " в дереве не встречается"
ИНАЧЕ
    ВЫВЕСТИ "Элемент ", D, " встречается ", cnt, " раз(а)"
КОНЕЦ ЕСЛИ
```

Результат выполнения программы



Задание 3

Изменить функцию добавления элементов для исключения добавления одинаковых символов.

Изменённая функция построения дерева (CreateTree)

```
ФУНКЦИЯ CreateTree(root, r, data):
    ЕСЛИ r = NULL ТО
        r = get_struct(data)
        ЕСЛИ root = NULL ТО
            ВЕРНУТЬ r
        КОНЕЦ ЕСЛИ
        ЕСЛИ data > root.data ТО
            root.left = r
        ИНАЧЕ
            root.right = r
        КОНЕЦ ЕСЛИ
        ВЕРНУТЬ r
    КОНЕЦ ЕСЛИ
    ЕСЛИ data = r.data ТО
        ВЕРНУТЬ root
    КОНЕЦ ЕСЛИ

    ЕСЛИ data > r.data ТО
        CreateTree(r, r.left, data)
    ИНАЧЕ
        CreateTree(r, r.right, data)
    КОНЕЦ ЕСЛИ

    ВЕРНУТЬ root
КОНЕЦ ФУНКЦИИ
```

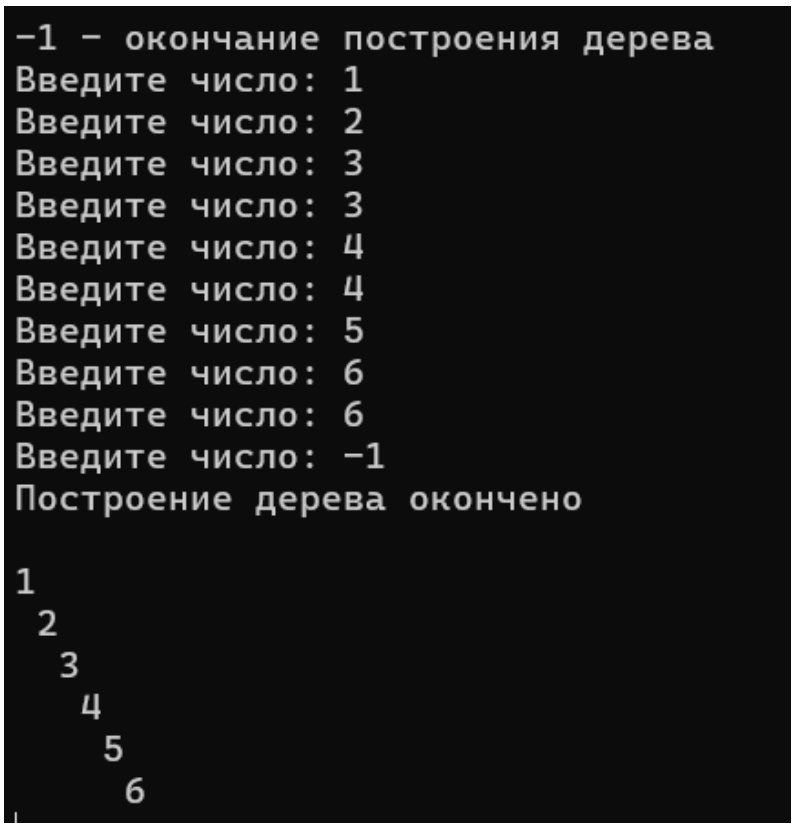
Фрагмент главной программы (ввод без дубликатов)

```
root = NULL
start = 1
ВЫВЕСТИ "-1 - окончание построения дерева"
```

```
ПОКА start = 1:
    ВЫВЕСТИ "Введите число: "
    СЧИТАТЬ число D
    ЕСЛИ D = -1 ТО
        ВЫВЕСТИ "Построение дерева окончено"
        start = 0
    ИНАЧЕ
        root = CreateTree(root, root, D)
    КОНЕЦ ЕСЛИ
КОНЕЦ ПОКА

print_tree(root, 0)
```

Результат выполнения программы



Задание 4

Оценить сложность процедуры поиска по значению в бинарном дереве.

Сложность поиска в бинарном дереве поиска равна высоте дерева, то есть количеству шагов от корня до самого дальнего листа.

Высота дерева зависит от его формы:

- Лучший случай. Дерево сбалансировано, и при увеличении числа узлов высота растёт очень медленно: чтобы вместить n узлов, нужно удваивать количество узлов на каждом уровне, поэтому высота — это то число, в которое нужно возвести двойку, чтобы получить n. Сложность поиска — O(log n).
- Средний случай. При случайном порядке ввода данных дерево получается достаточно ветвистым, и сложность поиска примерно такая же, как в лучшем случае — O(log n).
- Худший случай. Дерево вырождается в линейный список — это происходит, если данные поступают уже отсортированными. Сложность поиска — O(n).

Ключевой момент — высота дерева. В сбалансированном дереве высота логарифмическая, а в вырожденном — линейная. Поскольку поиск идёт по пути от корня к листу, его сложность напрямую зависит от высоты.

Вывод

В ходе лабораторной работы было реализовано бинарное дерево поиска на языке C. Освоены операции поиска значения, подсчёта вхождений элемента и добавления узлов с исключением дубликатов. Установлено, что сложность поиска зависит от высоты дерева: в сбалансированном случае — O(log n), в вырожденном — O(n). Получены навыки работы с рекурсивными структурами данных и динамической памятью.